Chapter 9

Objects & Classes

Objectives

9.1 To describe objects and classes, and use classes to model objects (§9.2).
9.2 To use UML graphical notation to describe classes and objects (§9.2).
9.3 To demonstrate how to define classes and create objects (§9.3).
9.4 To create objects using constructors (§9.4).
9.5 To access objects via object reference variables (§9.5).
9.6 To define a reference variable using a reference type (§9.5.1).
9.7 To access an object's data and methods using the object member access operator (**.**) (§9.5.2).
9.8 To define data fields of reference types and assign default values for an object's data fields (§9.5.3).
9.9 To distinguish between object reference variables and primitive data type variables (§9.5.4).
9.10 To use the Java library classes **Date**, **Random**, and **Point2D** (§9.6).
9.11 To distinguish between instance and static variables and methods (§9.7).
9.12 To define private data fields with appropriate **get** and **set** methods (§9.8).
9.13 To encapsulate data fields to make classes easy to maintain (§9.9).
9.14 To develop methods with object arguments and differentiate between primitive-type arguments and object-type arguments (§9.10).
9.15 To store and process objects in arrays (§9.11).
9.16 To create immutable objects from immutable classes to protect the contents of objects (§9.12).
9.17 To determine the scope of variables in the context of a class (§9.13).
9.18 To use the keyword **this** to refer to the calling object itself (§9.14).

Objectives

9.1 To describe objects and **classes**, and use classes to model **objects** (§9.2).
9.2 To use **UML** graphical notation to describe classes and objects (§9.2).
9.3 To demonstrate how to define classes and create objects (§9.3).
9.4 To create objects using **constructors** (§9.4).
9.5 To access objects via object reference variables (§9.5).
9.6 To define a reference variable using a reference type (§9.5.1).
9.7 To access an **object's data** and methods using the **object member access operator** (**.**) (§9.5.2).
9.8 To define **data fields** of reference types and assign default values for an object's data fields (§9.5.3).
9.9 To distinguish between object reference variables and primitive data type variables (§9.5.4).
9.10 To use the Java library classes **Date**, **Random**, and Point2D (§9.6).
9.11 To distinguish between **instance** and **static variables** and methods (§9.7).
9.12 To define private data fields with appropriate **get** and **set methods** (§9.8).
9.13 To encapsulate data fields to make classes easy to maintain (§9.9).
9.14 To develop methods with object arguments and differentiate between **primitive-type arguments** and **object-type arguments** (§9.10).
9.15 To store and process objects in arrays (§9.11).
9.16 To create **immutable objects** from **immutable classes** to protect the contents of objects (§9.12).
9.17 To determine the **scope of variables** in the context of a class (§9.13).
9.18 To use the keyword **this** to refer to the calling object itself (§9.14).

Objectives

10.1 To apply class abstraction to develop software (§10.2).

10.2 To explore the differences between the procedural paradigm and object-oriented paradigm (§10.3).

10.3 To discover the relationships between classes (§10.4).

10.4 To design programs using the object-oriented paradigm (§§10.5–10.6).

10.5 To create objects for primitive values using the wrapper classes (**Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, and **Boolean**) (§10.7).

10.6 To simplify programming using automatic conversion between primitive types and wrapper class types (§10.8).

10.7 To use the **BigInteger** and **BigDecimal** classes for computing very large numbers with arbitrary precisions (§10.9).

10.8 To use the **String** class to process immutable strings (§10.10).

10.9 To use the **StringBuilder** and **StringBuffer** classes to process mutable strings (§10.11).

Objectives

10.1 To apply class **abstraction** to develop software (§10.2).

10.2 To explore the differences between the **procedural paradigm** and **object-oriented paradigm** (§10.3).

10.3 To discover the relationships between classes (§10.4).

10.4 To design programs using the object-oriented paradigm (§§10.5–10.6).

10.5 To create objects for primitive values using the **wrapper classes** (**Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, and **Boolean**) (§10.7).

10.6 To simplify programming using automatic conversion between primitive types and wrapper class types (§10.8).

10.7 To use the **BigInteger** and **BigDecimal** classes for computing very large numbers with arbitrary precisions (§10.9).

10.8 To use the **String** class to process immutable strings (§10.10).

10.9 To use the **StringBuilder** and **StringBuffer** classes to process mutable strings (§10.11).

Object-oriented programming (OOP)

- Programming paradigm that uses "**objects**"
  to model real-world entities and concepts in code.

- **Objects** are created from **classes**,
  which define their **attributes** and **behavior**,
  and can interact with each other to solve problems.

- Large-scale software:
  complex, modular, reusable and maintainable code

Procedural Programming

Remember to bring a reusable grocery bag with you.

Double-check the list to make sure you get everything you need.

Don't forget to grab some fresh fruits and vegetables.

Check for any sales or deals before you start shopping.

Procedural Programming

Remember to bring a reusable grocery bag with you to the store.

Double-check your shopping list before starting to shop.

Don't forget to purchase some fresh fruits and vegetables.

Look for any sales or discounts before you start shopping.

If you're not sure where to find something,  ask an employee for help.

Check the expiration dates of the products you're purchasing.

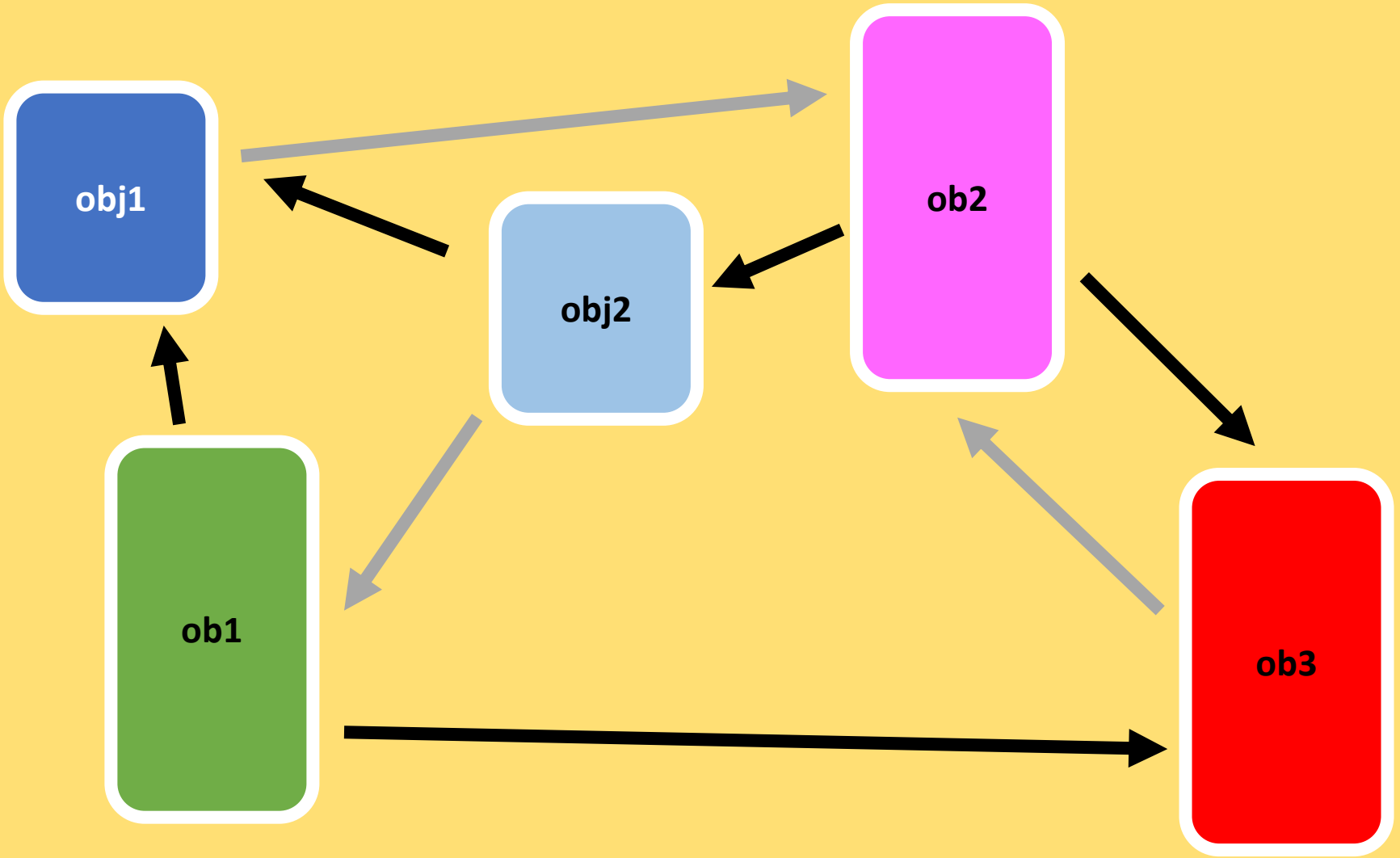
If you need assistance carrying your groceries,  ask someone for help.

Check the nutritional information and ingredients of packaged or canned goods.

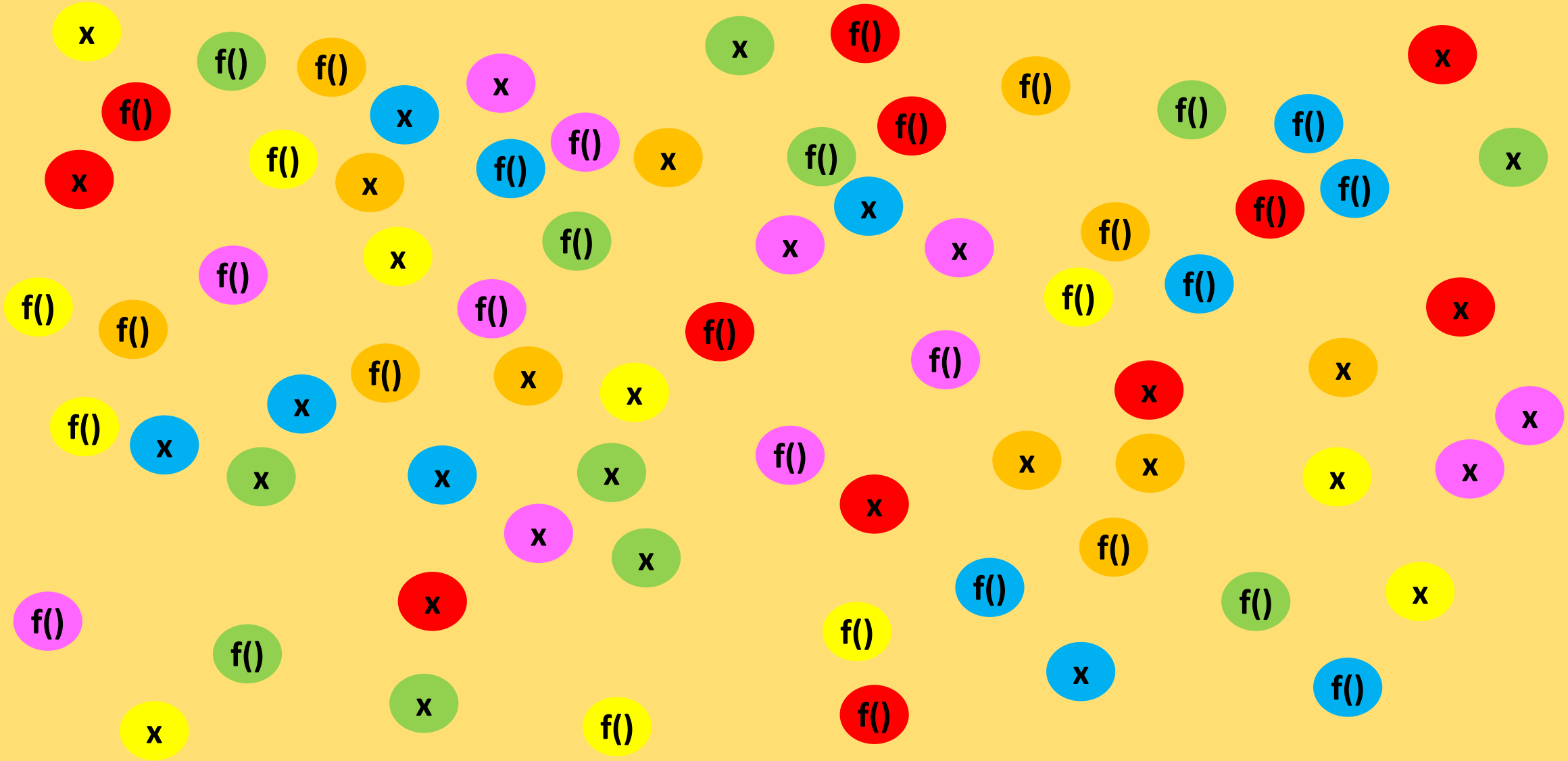
Be courteous to other shoppers by being mindful of your space and not blocking aisles.

Check the temperature of refrigerated items before purchasing.

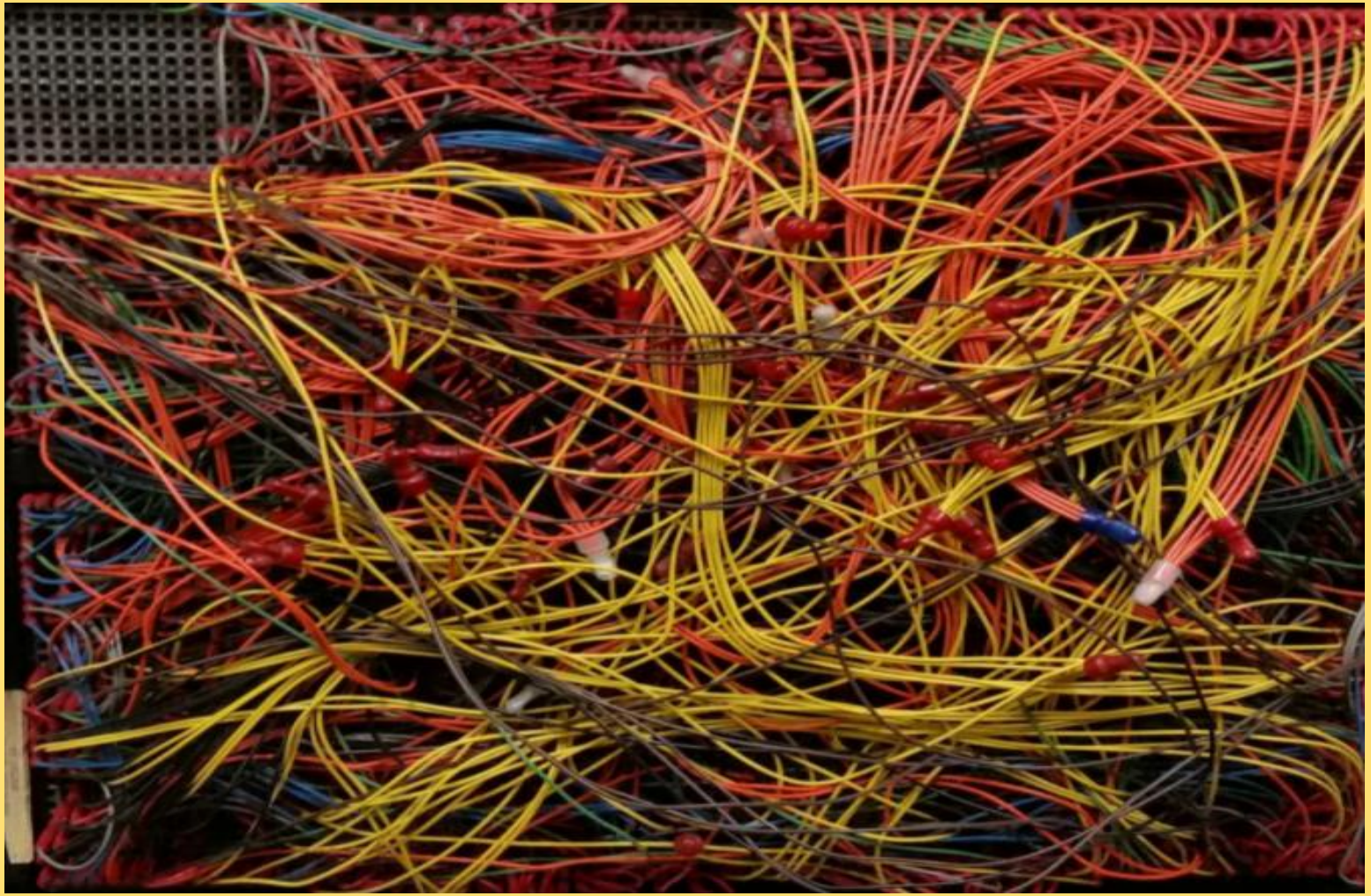
Object-oriented programming (OOP)



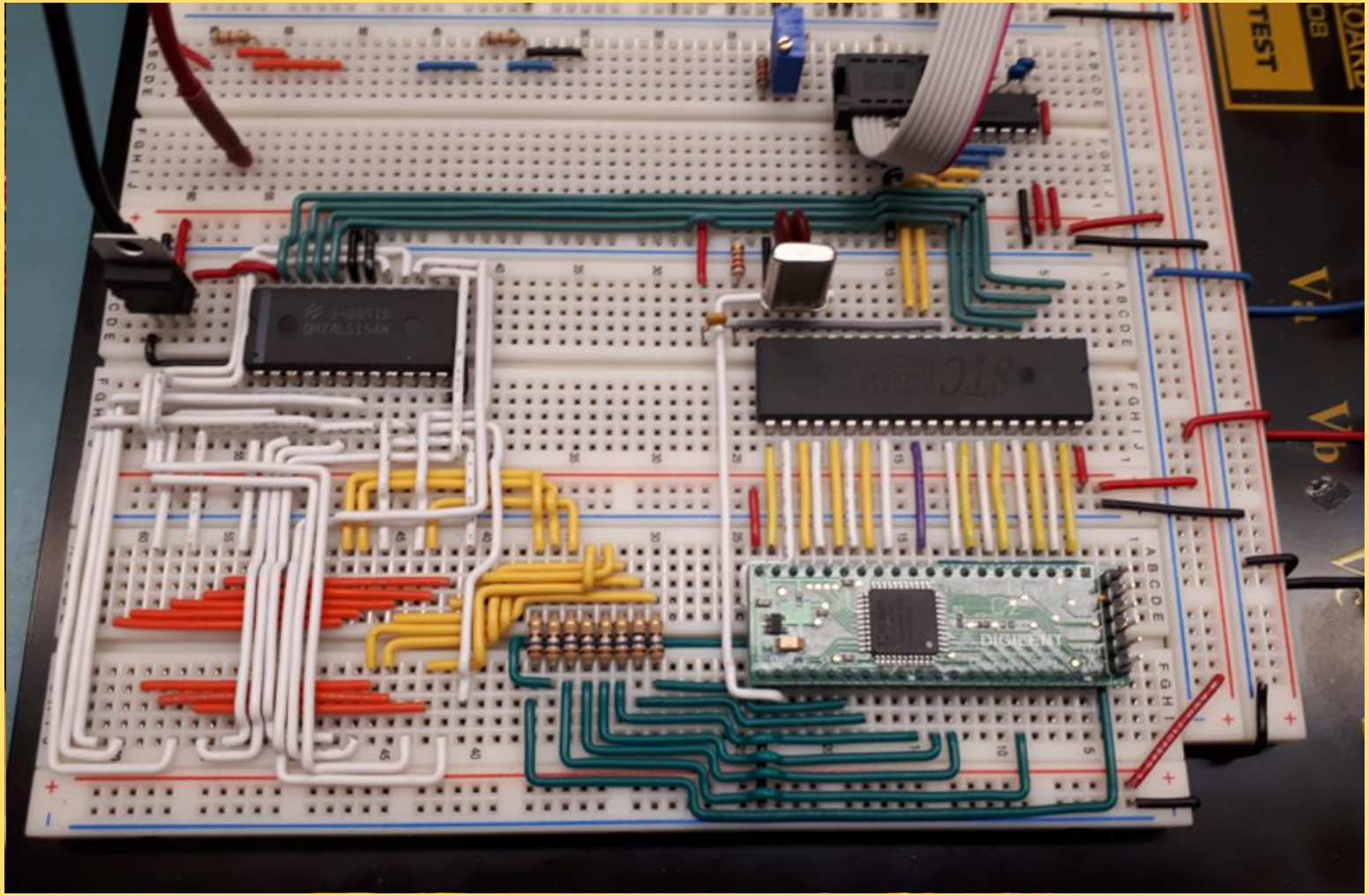
Procedural Programming gone wrong



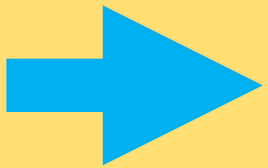
Spaghetti code



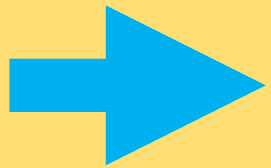
Object-oriented programming (OOP)

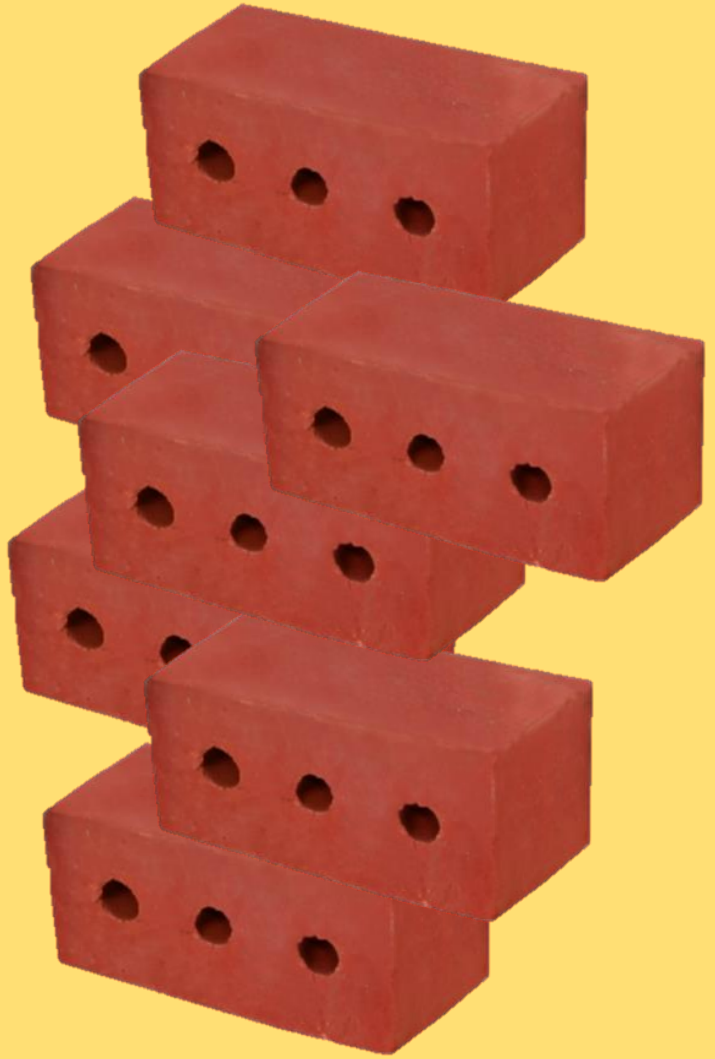


How to make an object?

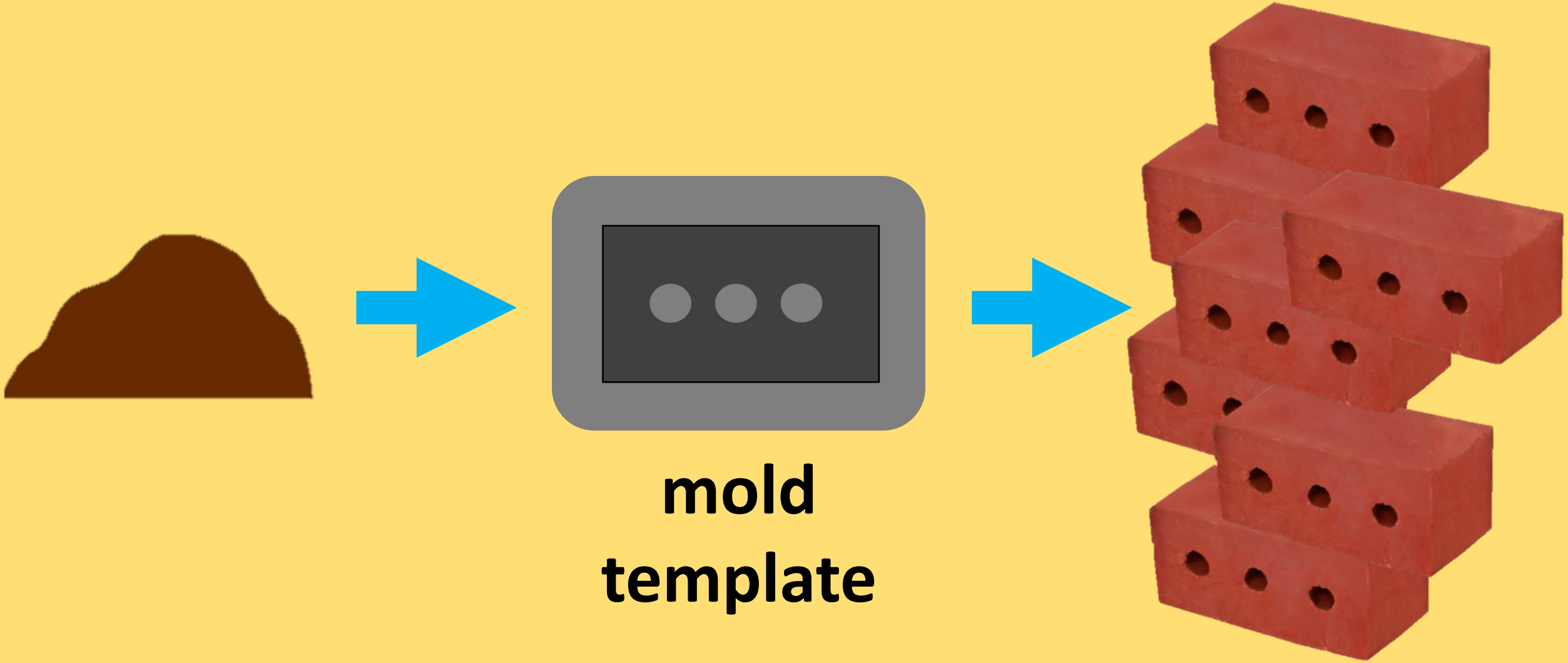


How to make many objects?



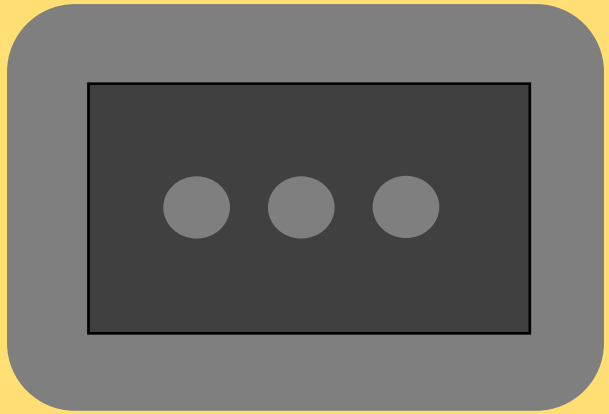


How to make many objects?

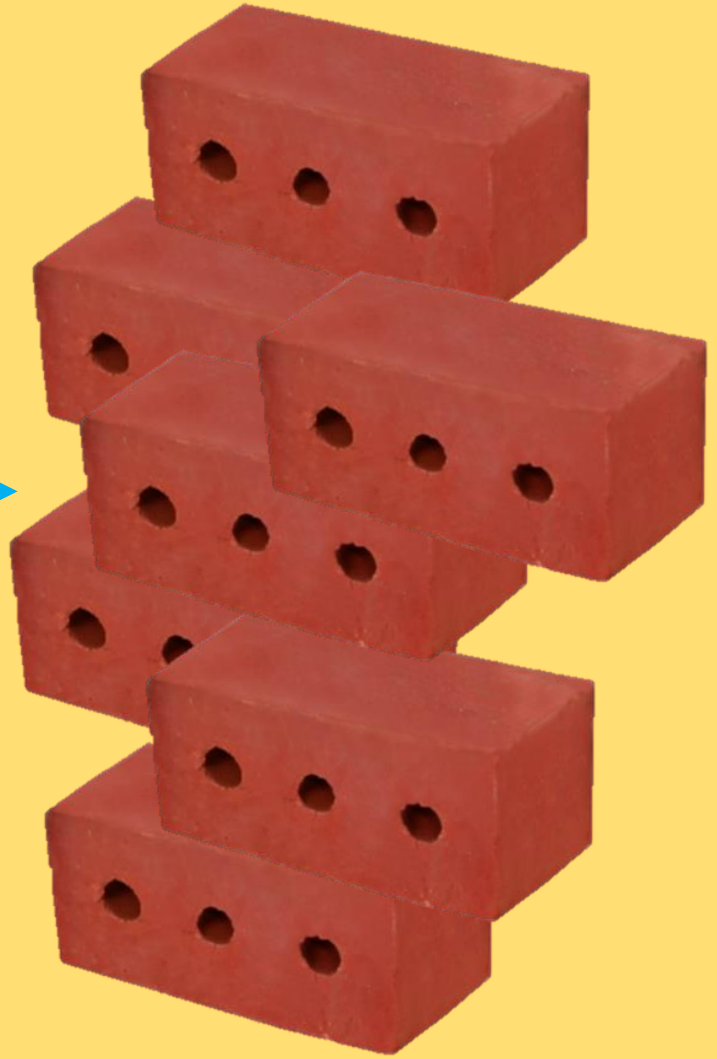


mold
template

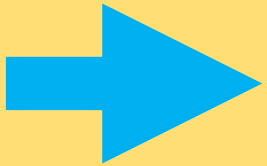
Class → Object

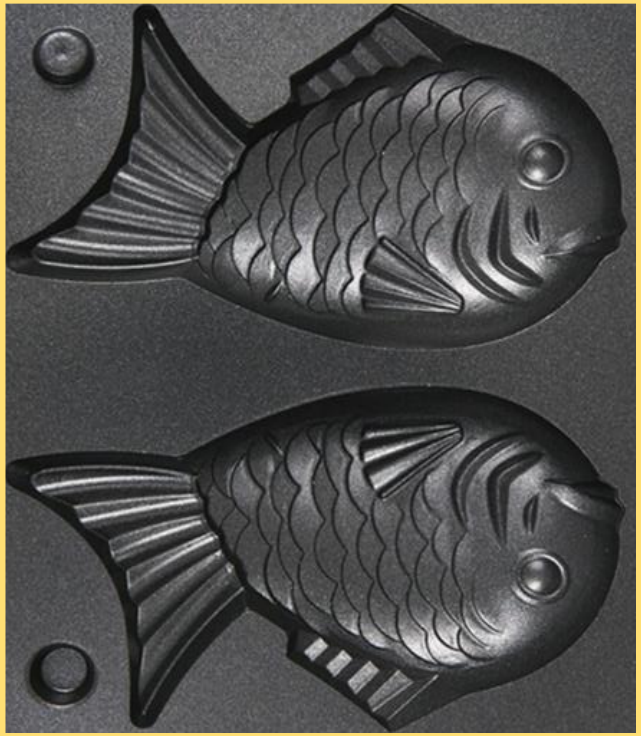


mold
template



Waffle vs. Waffle Iron



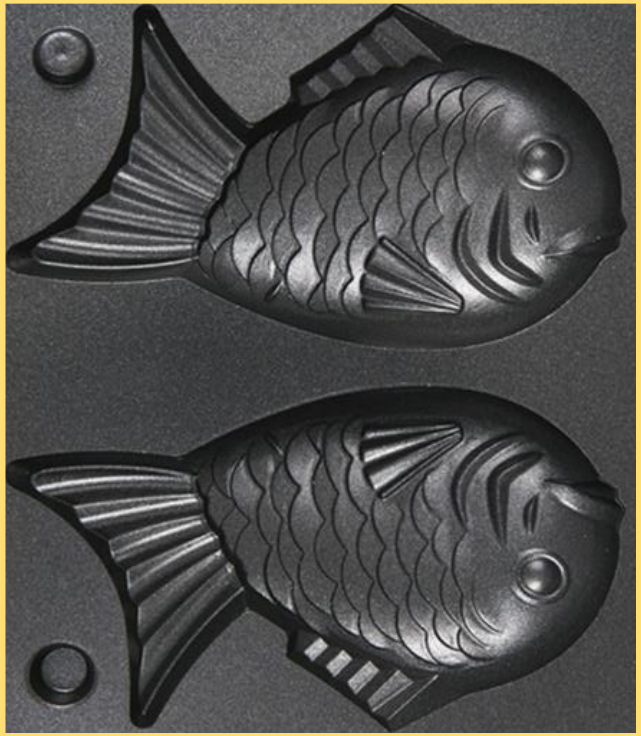


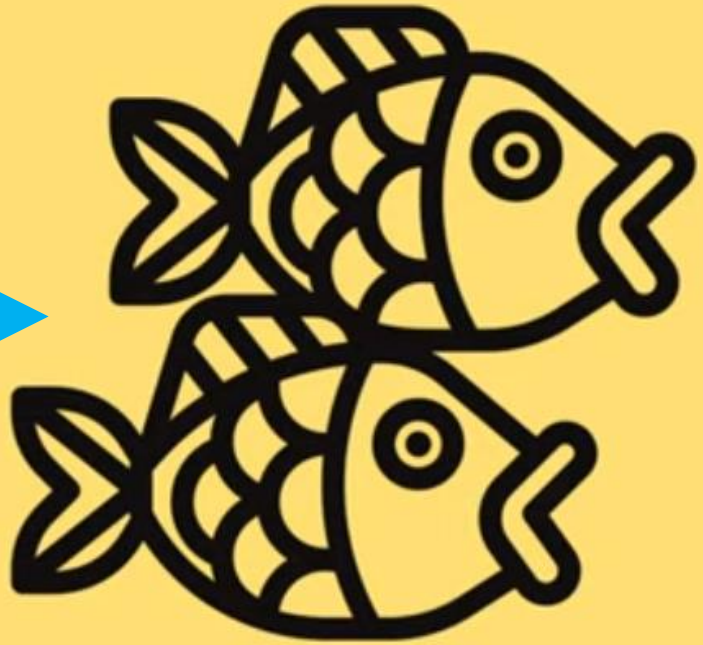
https://youtu.be/IfofLeL6duQ?t=348

Waffle vs. Waffle Iron

https://youtu.be/IfofLeL6duQ?t=348

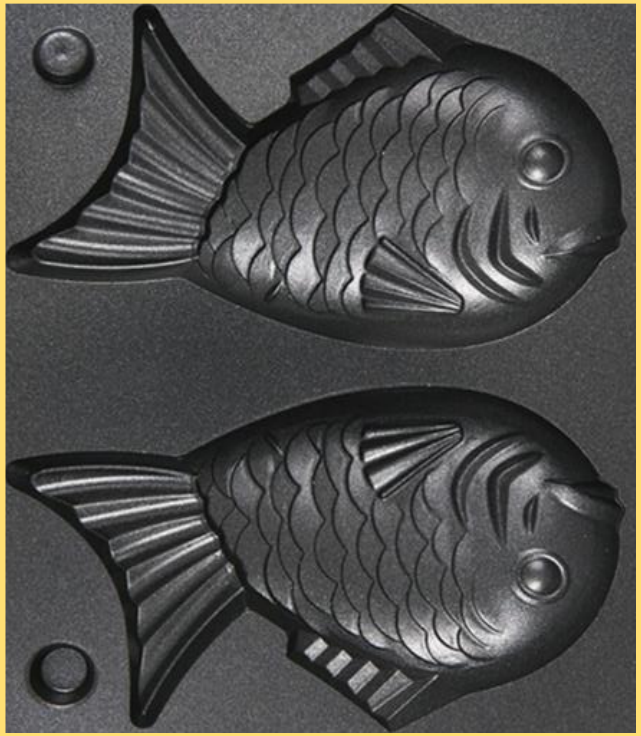
Waffle vs. Waffle Iron

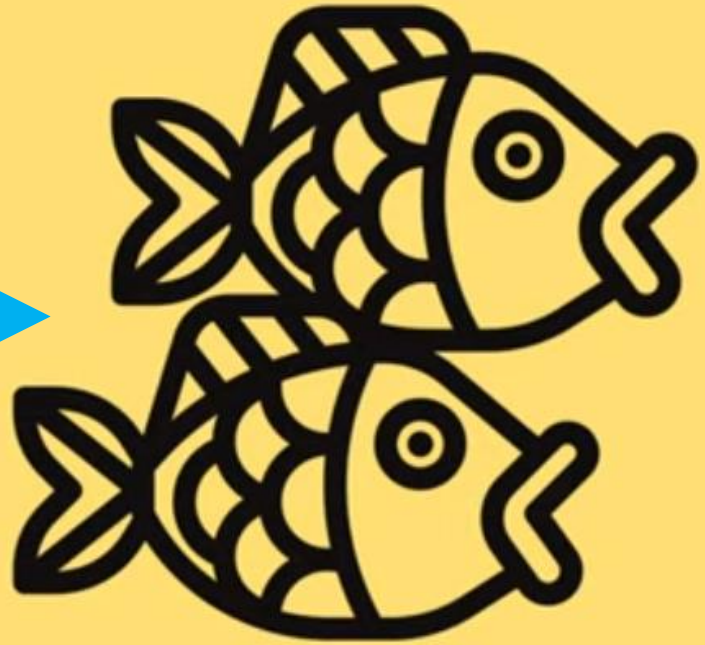




https://youtu.be/IfofLeL6duQ?t=348

Class → Object





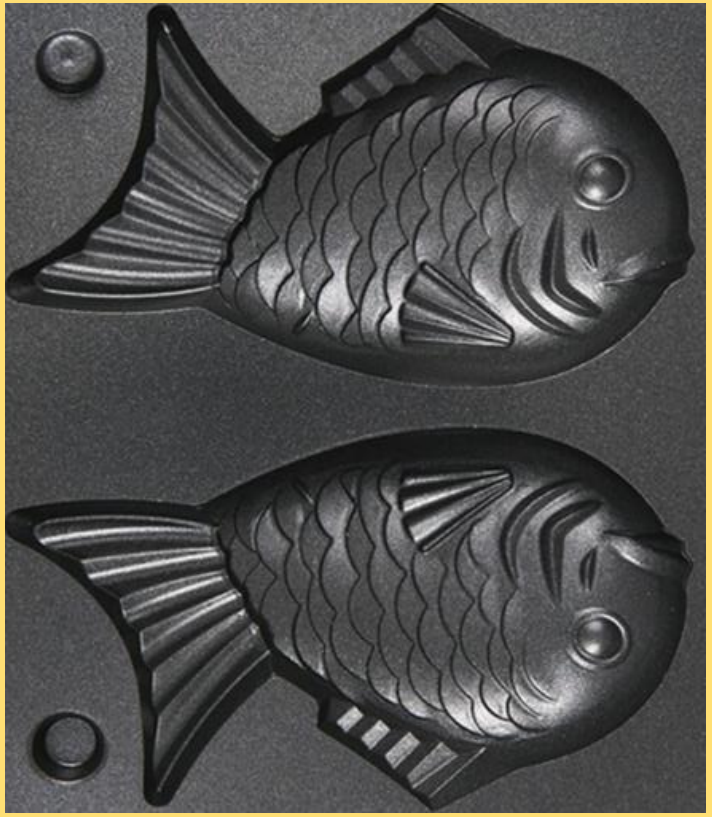
https://youtu.be/IfofLeL6duQ?t=348

Class

- **template** or **blueprint** that defines
  the **properties** & **behaviors** of objects

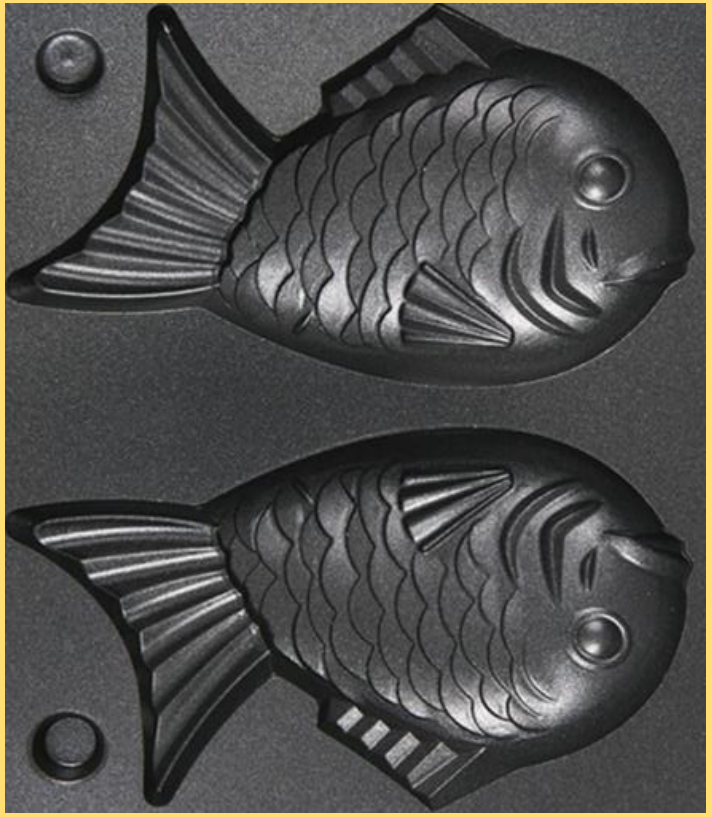
- defined once

- stores no (instance) data



Class

- **template** or **blueprint** that defines
  the **properties** & **behaviors** of objects
- combination of
  - ✓ **variables** & **methods**



Object

- **instance** of a class

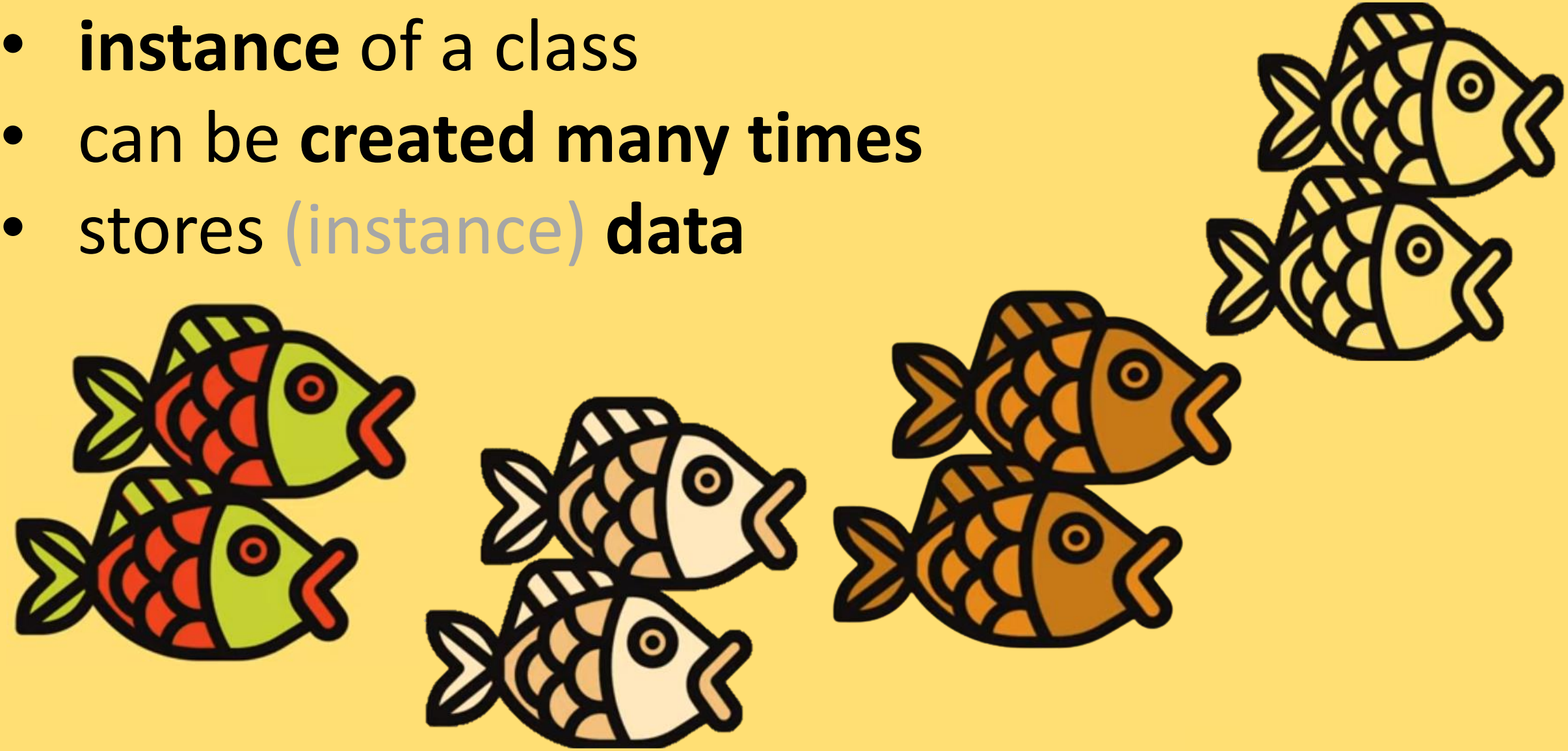
instantiation

***instantiate* an *object* =**

*create an **object** from a **class***

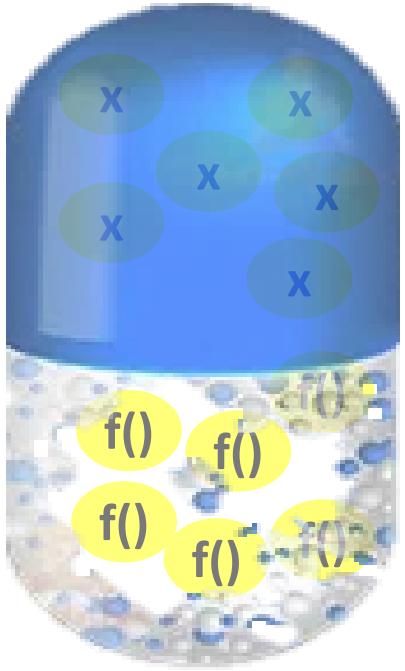
Object

- **instance** of a class
- can be **created many times**
- stores (instance) **data**



Object-oriented programming (OOP)

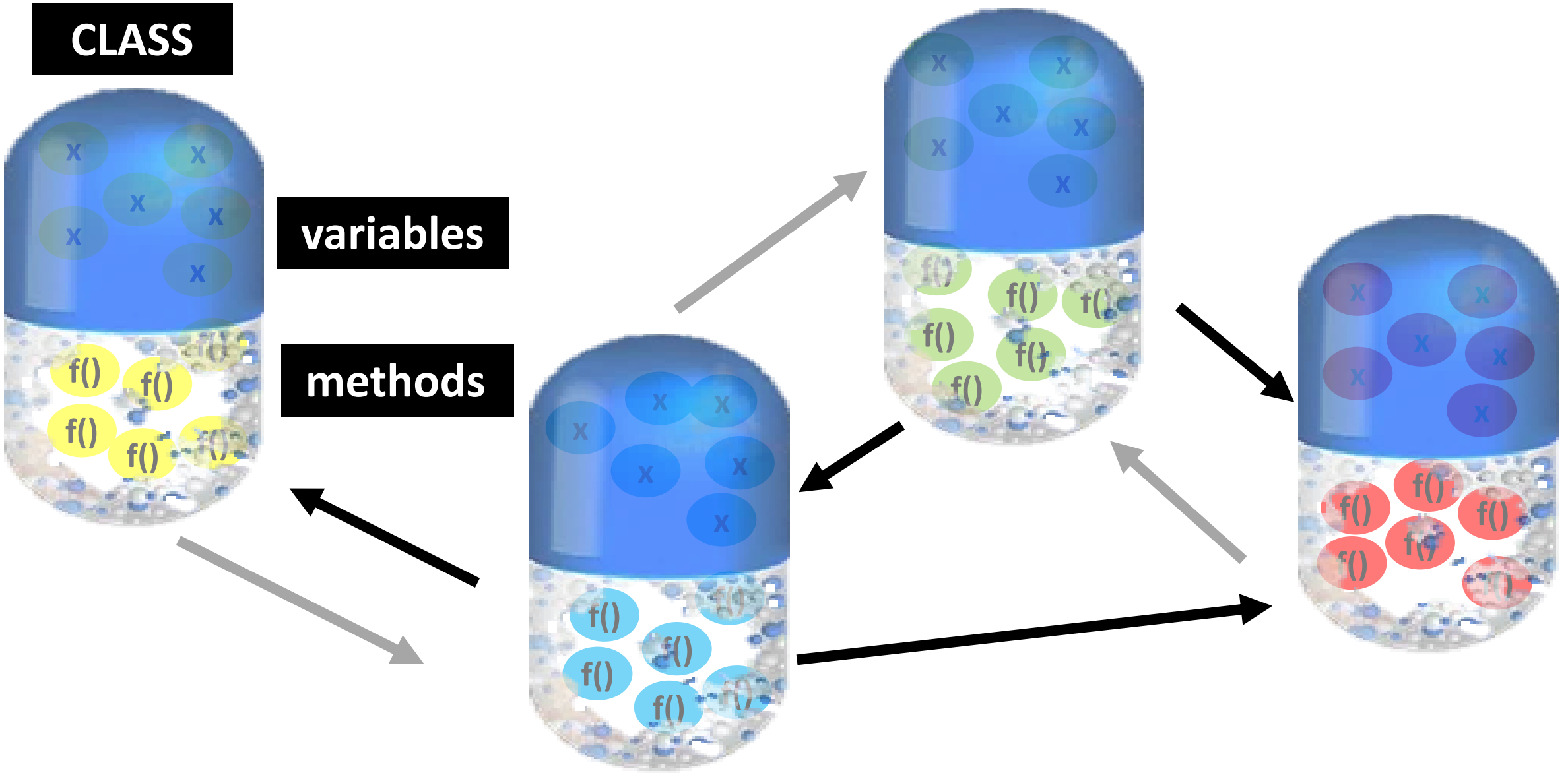
CLASS



variables

methods

Object-oriented programming (OOP)



Object

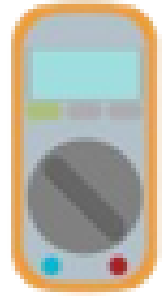
```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        if(number % 2 == 0) {
            System.out.println(number + " is even");
        } else {
            System.out.println(number + " is odd");
        }
    }
}
```

1

2

3

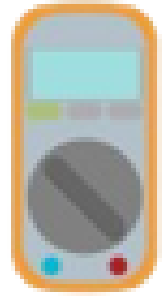
what you need



class

object

method



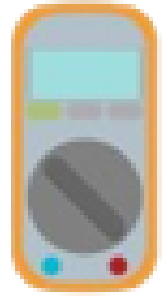
code in file

code in Memory

class

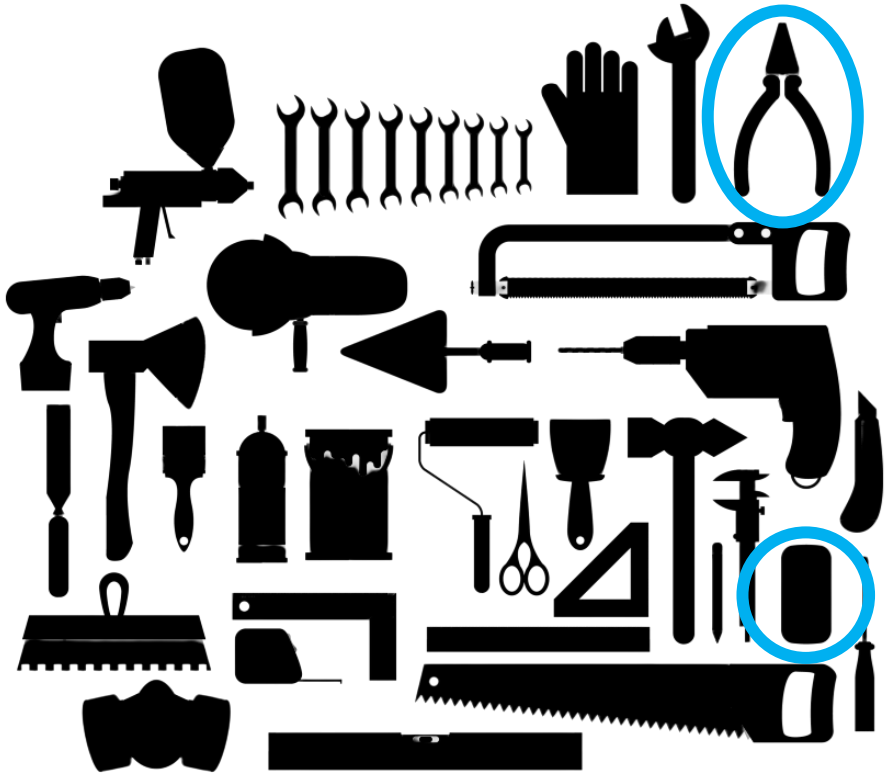
object

method

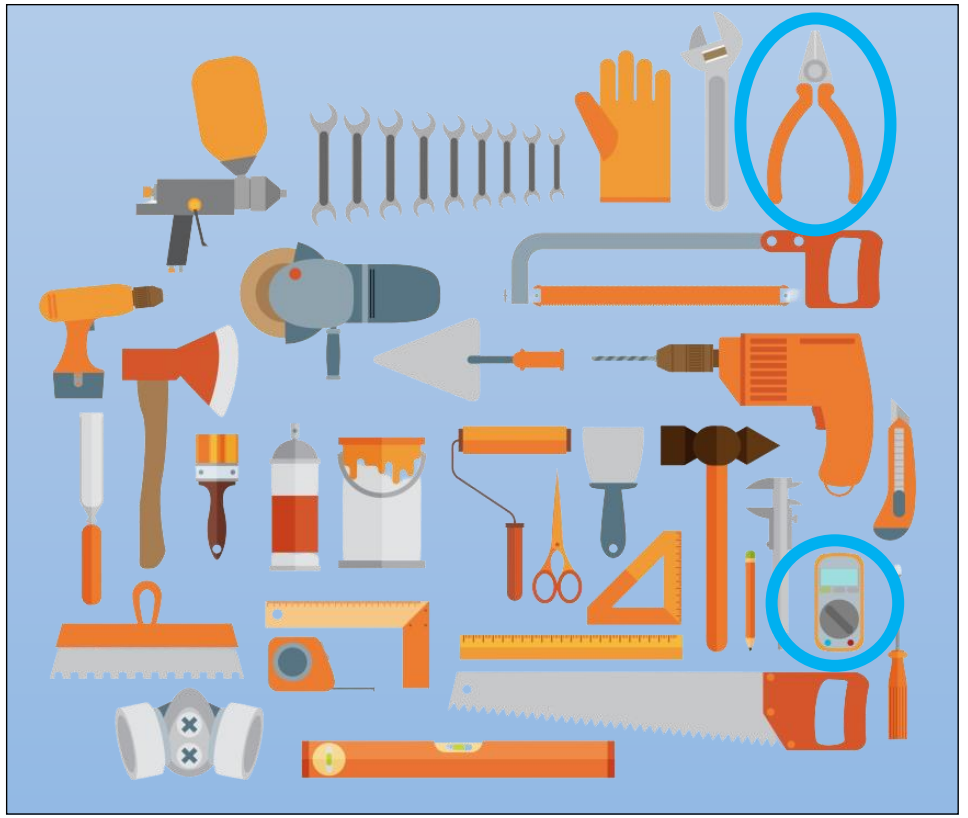


```
int *** = obj.nextInt();
String *** = obj.nextLine();
```

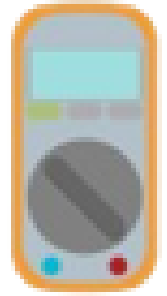
class



object



method

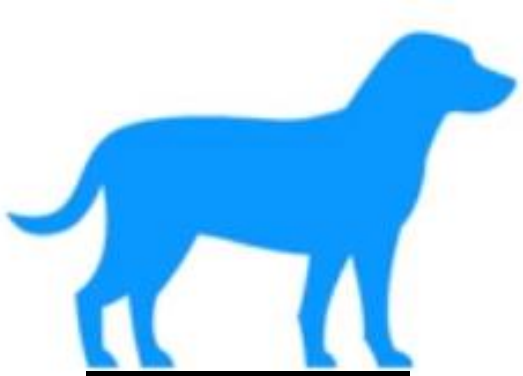


```
import java.util.Scanner;
```

```
Scanner obj = new Scanner(System.in);
```

```
int *** = obj.nextInt();
String *** = obj.nextLine();
```

Class → Object



Class

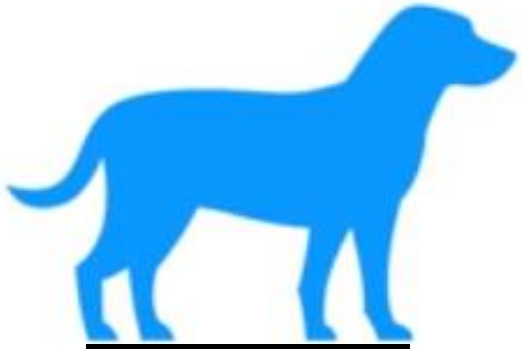
DOG
name breed age bark
say()

Variable/ Field/ Property/ Attribute

Method/Function

Object

Class → Object



Class

UML class diagram

DOG
name breed age bark
say()

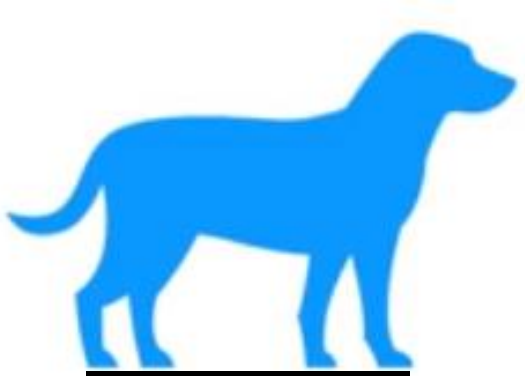
Name

Variable/ Field/ Property/ Attribute

Method/Function

Object

Class → Object



Class

DOG
name breed age bark
say()

Object

name= “Bill”
breed= “Bulldog”
bark= “ruff ruff”
age= 3
say() “Bill says ruff ruff”

name= “Sally”
breed= “German Shepherd”
bark= “woof woof”
age= 1
say() “Sally says woof woof”

Class

CLASS

```
// Class Declaration
class Dog {
    // Properties




    // methods



}
```

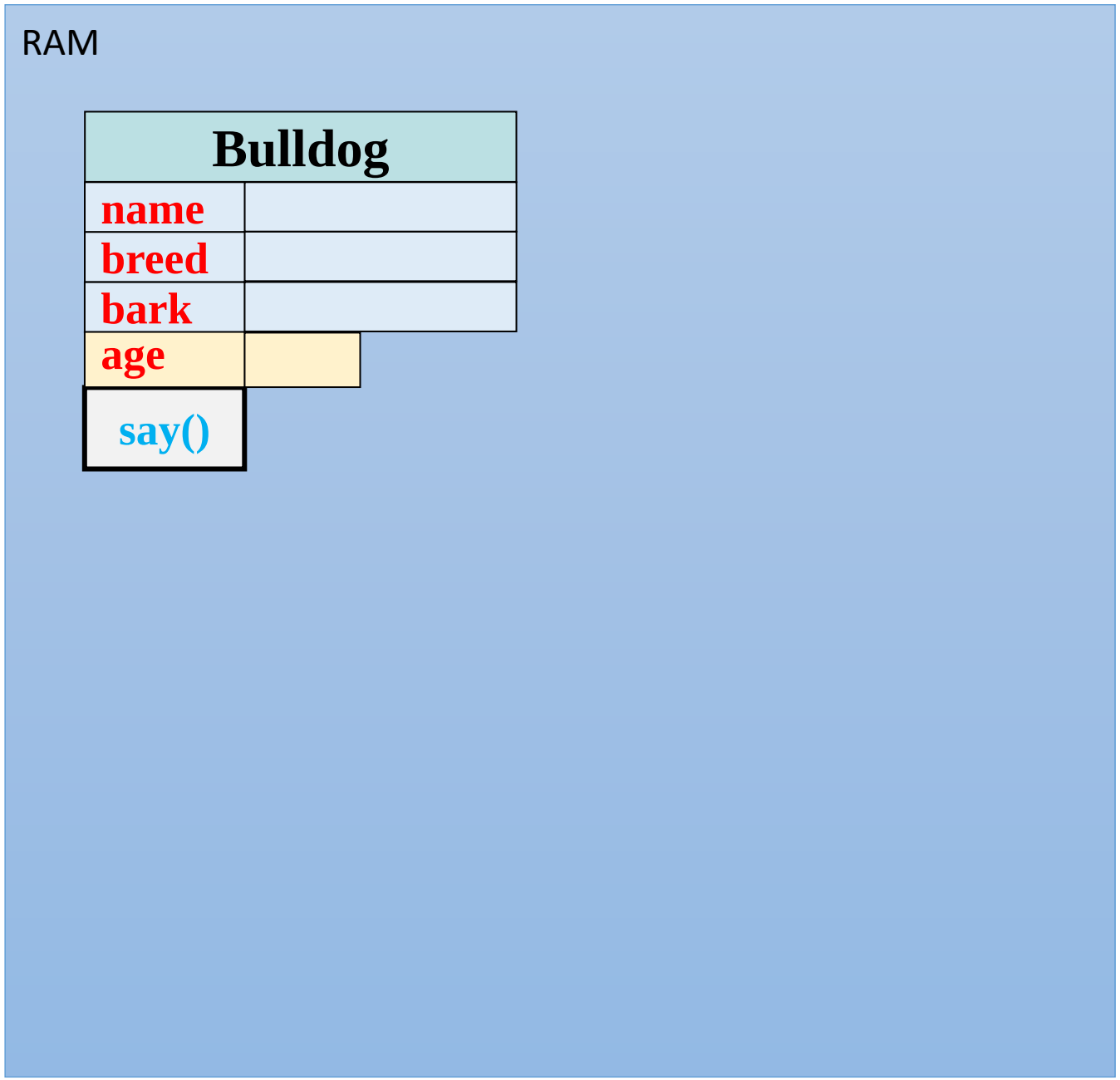
Class

CLASS

```
// Class Declaration
class Dog {
    // Properties
    String name;
    String breed;
    String bark;
    int age;

    // methods
    public void say() {
        System.out.println(name + " says: " + bark);
    }
}
```

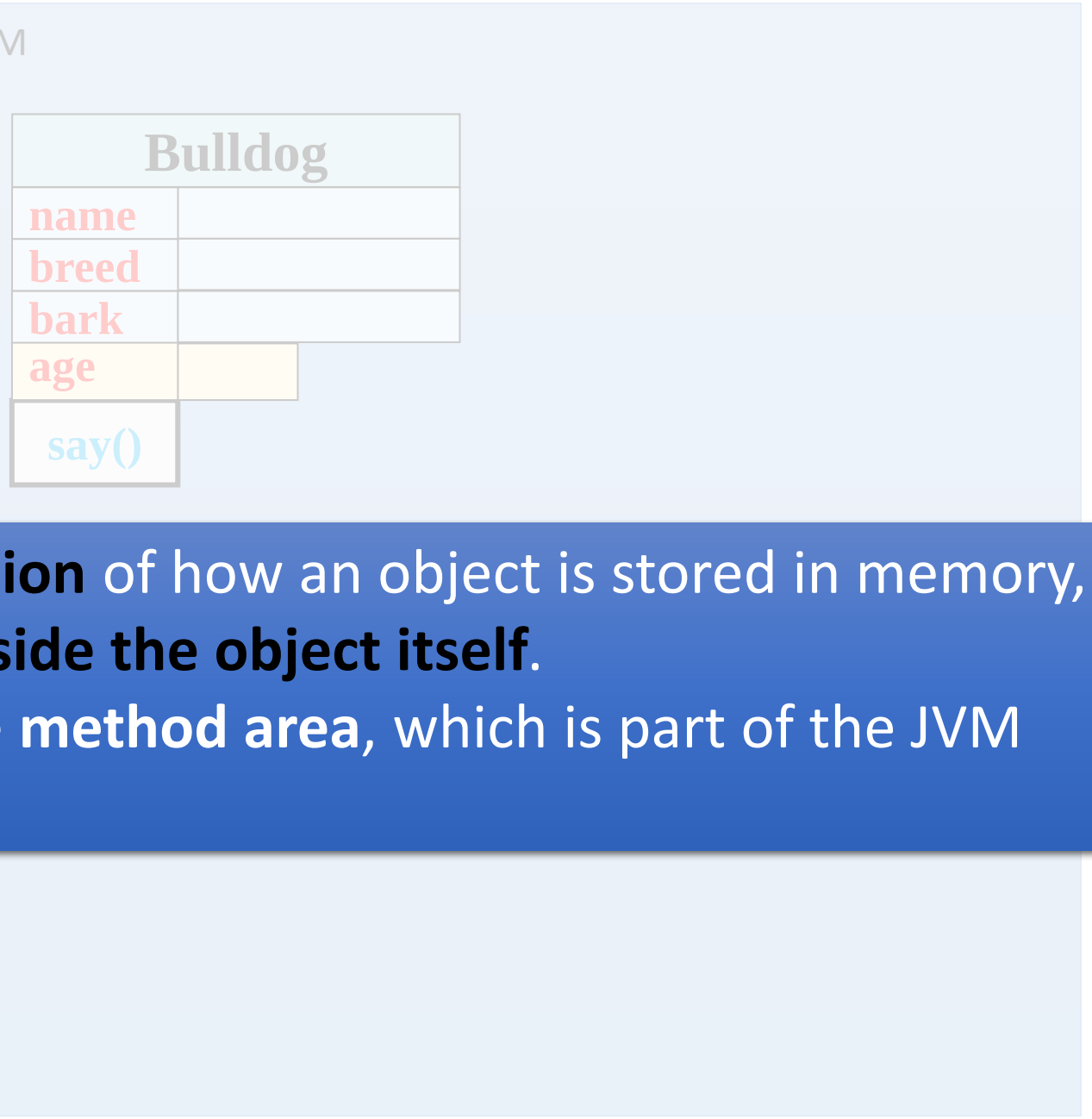
Object

```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();


        //Object 1 properties ================



        //Object 2 properties ==================


        //Object 1 method

    }
}
```



Object

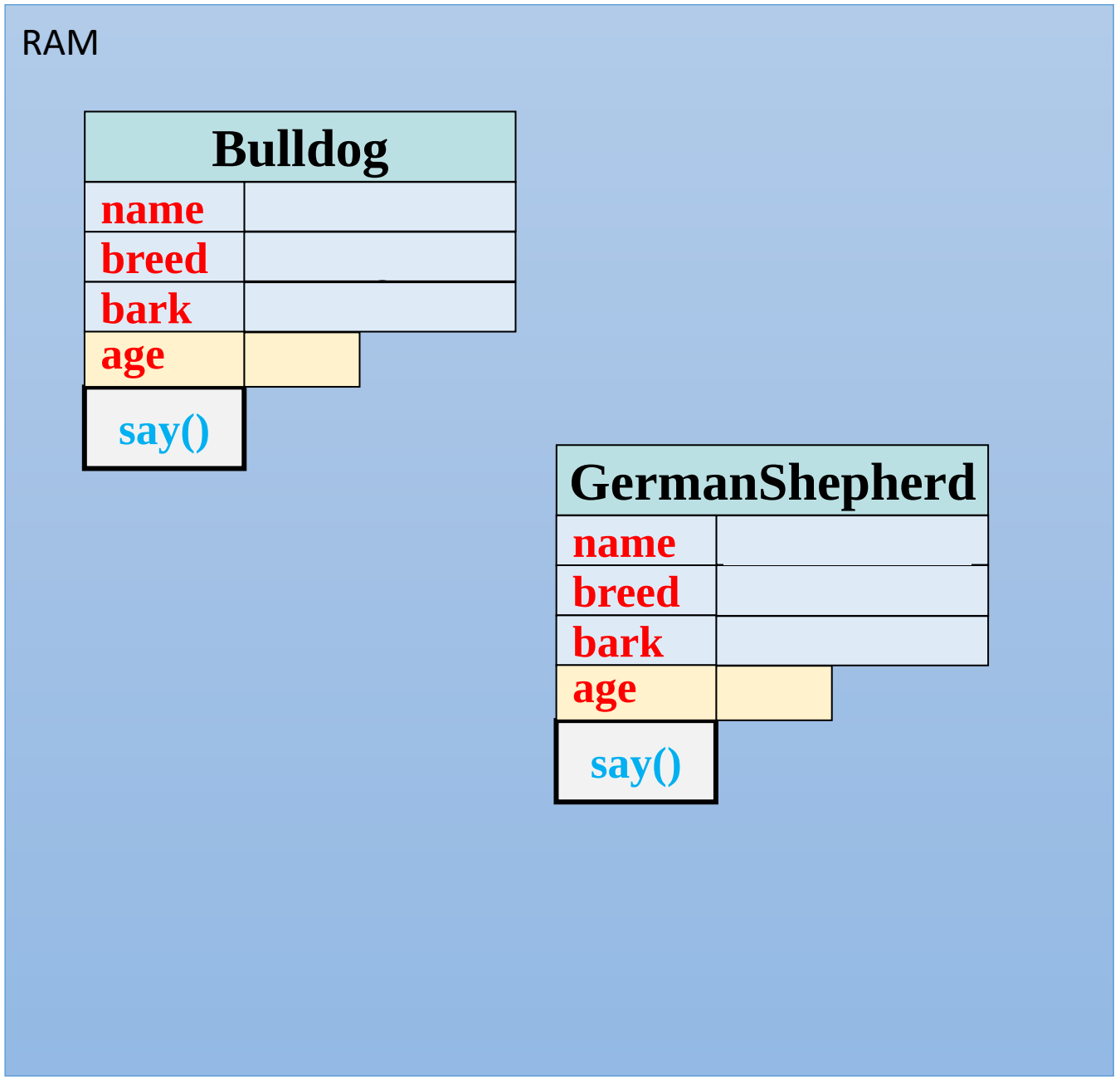
```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();

        //Object 1 properties ================
    }
}
```



This diagram is a **conceptual representation** of how an object is stored in memory, but in reality, **methods are not stored inside the object itself**.
Instead, they are stored separately in the **method area**, which is part of the JVM runtime memory.

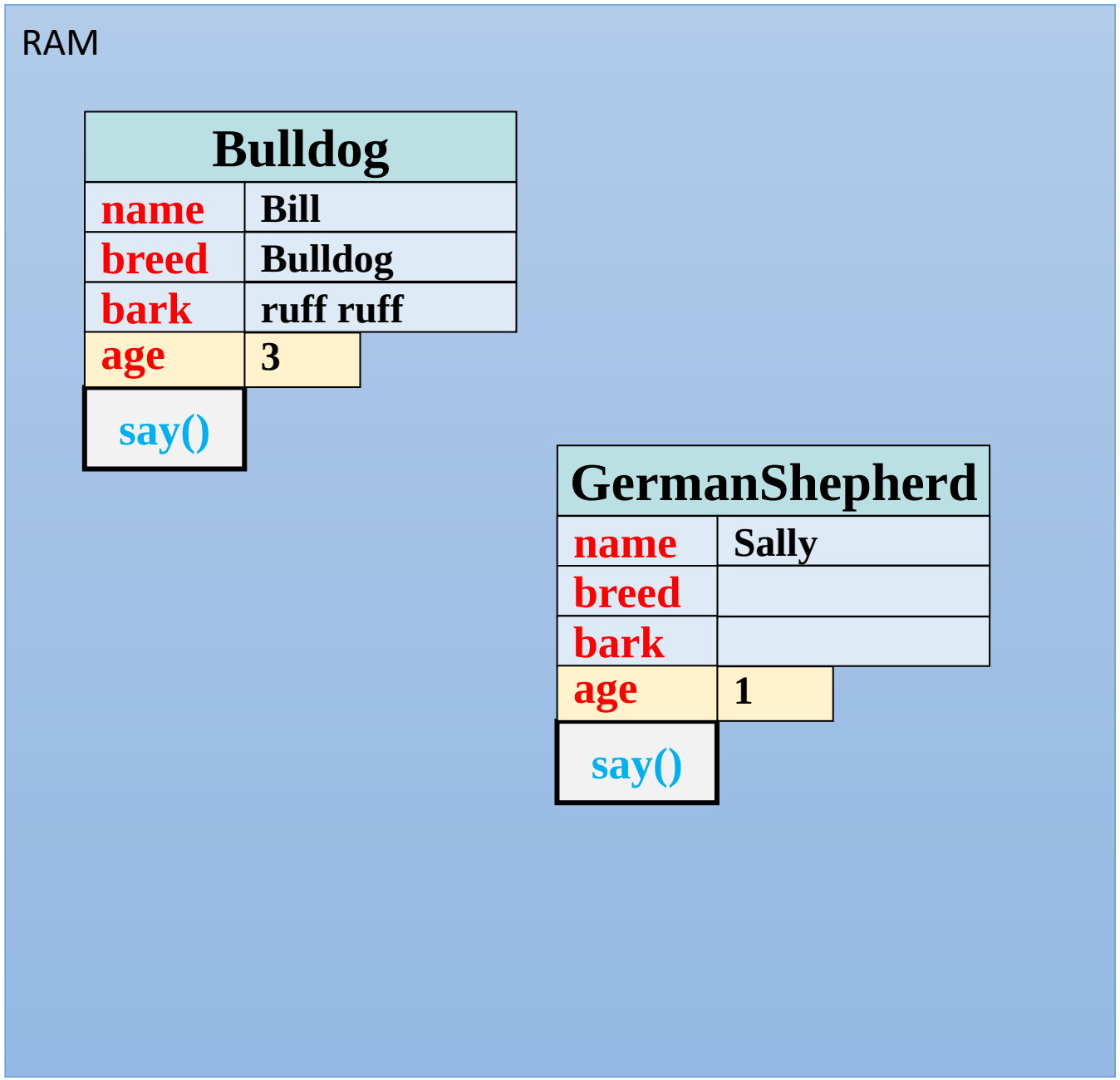
Object

```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================



        //Object 2 properties ==================


        //Object 1 method

    }
}
```



Object

```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================
        bulldog.name="Bill";
        bulldog.breed="Bulldog";
        bulldog.bark="ruff ruff";
        bulldog.age=3;

        //Object 2 properties ==================
        GermanShepherd.name="Sally";
        GermanShepherd.age=1;

        //Object 1 method

    }
}
```



Object

```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================
        bulldog.name="Bill";
        bulldog.breed="Bulldog";
        bulldog.bark="ruff ruff";
        bulldog.age=3;

        //Object 2 properties ==================
        GermanShepherd.name="Sally";
        GermanShepherd.age=1;

        //Object 1 method
        bulldog.say();
    }
}
```

RAM

Bulldog	
name	Bill
breed	Bulldog
bark	ruff ruff
age	3
say()	

GermanShepherd	
name	Sally
breed	
bark	
age	1
say()	

Object

```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================
        bulldog.name="Bill";
        bulldog.breed="Bulldog";
        bulldog.bark="ruff ruff";
        //Object 2 properties ================
        GermanShepherd.name="Sally";
        GermanShepherd.age=1;

        //Object 1 method
        bulldog.say();
    }
}
```

```
public void say() {
    System.out.println(name + " says: " + bark);
}
```

RAM

Bulldog	
name	Bill
breed	Bulldog
bark	ruff ruff
age	3
say()	

GermanShepherd	
name	Sally
breed	
bark	
age	1
say()	

Bill says: ruff ruff

Class Declaration

Dog.java

```java
// Class Declaration
public class Dog {
    // Properties
    String name;
    String breed;
    String bark;
    int age;

    // methods
    public void say() {
        System.out.println(name + " says: " + bark);
    }
}
```

Class Declaration

Main.java

```
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================
        bulldog.name="Bill";
        bulldog.breed="Bulldog";
        bulldog.bark="ruff ruff";
        bulldog.age=3;

        //Object 2 properties ================
        GermanShepherd.name="Sally";
        GermanShepherd.age=1;

        //Object 1 method
        bulldog.say();

    }
}
```

Dog.java

```
// Class Declaration
public class Dog {
   // Properties
   String name;
   String breed;
   String bark;
   int age;

   // methods
   public void say() {
       System.out.println(name + " says: " + bark);
   }
}
```

Class Declaration

Main.java

```java
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================
        bulldog.name="Bill";
        bulldog.breed="Bulldog";
        bulldog.bark="ruff ruff";
        bulldog.age=3;

        //Object 2 properties ================
        GermanShepherd.name="Sally";
        GermanShepherd.age=1;

        //Object 1 method
        bulldog.say();

    }
}
```

```java
// Class Declaration
public class Dog {
    // Properties
    String name;
    String breed;
    String bark;
    int age;

    // methods
    public void say() {
        System.out.println(name + " says: " + bark);
    }
}
```

Class Declaration

Main.java

```java
public class Main{
    public static void main(String[] args) {
        //Objects
        Dog Bulldog = new Dog();
        Dog GermanShepherd = new Dog();

        //Object 1 properties ================
        bulldog.name="Bill";
        bulldog.breed="Bulldog";
        bulldog.bark="ruff ruff";
        bulldog.age=3;

        //Object 2 properties ==================
        GermanShepherd.name="Sally";
        GermanShepherd.age=1;

        //Object 1 method
        bulldog.say();

    }
}
```

```java
// Class Declaration
        class Dog {
   // Properties
    String name;
    String breed;
    String bark;
    int age;

    // methods
    public void say() {
        System.out.println(name + " says: " + bark);
    }
}
```